

Apskaita5 ERP Modernization — Case Study

Oct 5, 2026 · @Kirill Fuchs

Executive summary

Recommendation: rebuild Apskaita5 as one multi-tenant cloud product with AI-assisted modernisation, and migrate customers one by one behind a financial-equivalence safety net. The plan meets all five business goals:

- first customers live in month 5;
- 2,970 customers (70.7 percent) by month 12;
- run cost per migrated customer down 55 percent at month 12, and 59 percent across all customers once legacy is switched off in month 20;
- statutory changes in production in under 3 weeks (20 days at most);
- zero financial regression, enforced by a 100 percent differential-test gate and reconciliation to the cent at every cutover.

Why the business hurts. A full read of the reference code explains each pain point:

- **Linear cost.** Each of 4,200 customers has its own database, so infrastructure cost grows with every customer.
- **Slow statutory change.** Every statutory form version is a copied class with hardcoded dates, and every change must be built into 310 forks and wait for one of two releases a year. That is the 10 to 14 weeks.
- **No safety net.** About 522,000 lines of VB.NET on a 2006-era framework have no tests, so every change is risky and maintenance absorbs 62 percent of capacity.

What we do. Five moves, each detailed below:

1. **One platform.** A modular monolith on .NET 10, pooled PostgreSQL with row-level security in cells of about 600 tenants, and a dedicated database for the largest 3 percent.
2. **Statutory rules as data.** Signed, effective-dated rule packs ship outside the code release train in about 10 working days.
3. **No more forks.** The 310 forks collapse into configuration, templates and rules, or out-of-process extensions.
4. **AI at four layers, never unchecked.** L0 builds a rule inventory, L1 translates calculation code, L2 maps and migrates data, and L3 proves equivalence. At every layer AI drafts, tools verify and people sign.

5. Tenant-by-tenant migration. Six waves (W0 to W5), one-night cutovers, and rehearsed rollback until the first accepted filing.

The product gains four AI features: invoice and bank statement extraction, anomaly detection, guarded natural-language reporting and a grounded support assistant. Each has written limits on what it may do alone.

Three findings from the code that change the plan:

- **Rounding disagrees with itself.** Money is stored as binary floating point, and five copies of the rounding function disagree: by emulation, about 1 in 550 VAT calculations differ by a cent between VB and SQL. L3 therefore compares to the cent and keeps a signed deviation register.
- **Customers type in their own tax rates.** Every company enters its own rates and the non-taxable income formula. Central rule packs remove a whole class of errors and tickets.
- **A clean seam exists.** Business logic is almost free of UI code and all SQL sits in one named dictionary. The legacy layer can run headless as the test oracle, and the rule inventory starts from a complete map.

Money. About 3.95M EUR over two years buys about 2.2M EUR a year in cash savings from year 3. Payback comes in month 39 on cash, or month 35 counting freed capacity. Five-year net cash is 3.96M EUR, and NPV at 8 percent is 2.4M EUR. Engineering time spent on maintenance falls from 62 percent to 30 percent or less, and tickets fall by about 40 percent.

First eight weeks.

- Confirm the 15 assumptions.
- Run the AI fork analysis and the L0 inventory for Ledger and VAT.
- Stand up the headless legacy harness.
- Pass programme gate P1.
- Rotate the plaintext database credential found in the shipped web configuration, today.

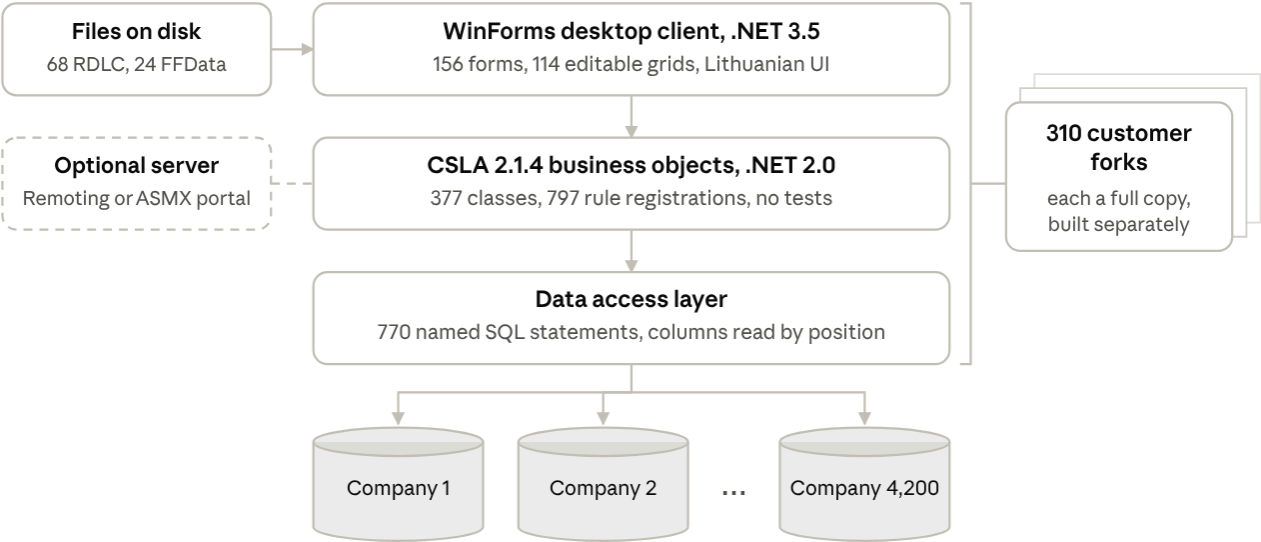
Current-state assessment

The reference code base is about 522,000 lines of VB.NET in 1,109 files, on .NET Framework 2.0 and 3.5 and CSLA 2.1.4, a 2006-era stack. It has no automated tests, stores money as binary floating point, defines no foreign keys, and implements each statutory form version as a copied class. Two things make modernisation tractable, though. Business logic is cleanly separated from the UI. All SQL lives in a named dictionary of 770 statements, which an AI-assisted inventory can read end to end.

These facts come from a full read of the public Apskaita5 repository. The firm's own version (about 3,000 files, per the brief) and its 310 forks are analysed in L0.

Today: one desktop stack, copied 310 times, over 4,200 separate databases

Current architecture as found in the reference code; dashed parts are optional



One MySQL database per company: 67 tables, money in DOUBLE, no foreign keys

Every customer has its own database, and every fork is a copy of the whole stack. Cost therefore grows with each customer, and every change has to be built 310 times.

What the code contains

Layer	Project	.vb files	Lines	What it holds
Business objects	ApskaitaObjects	537	304,727 (about 222,000 hand-written)	377 CSLA classes, 425 data-portal methods, 797 validation-rule registrations, 28 declaration classes
Desktop forms	AccDataBindingsWinForms	332	147,860 (101,658 designer-generated)	129 of the 156 forms; 114 editable grids with 1,846 columns

Layer	Project	.vb files	Lines	What it holds
Controls	AccControlsWinForms	82	32,624	Keyboard-friendly pickers, numeric and date boxes, report printing
Data access	AccDataAccessLayer	68	16,625	Named-SQL dictionary, ambient transactions, schema compare and fix, users and roles
Shell, reports, templates	Apskaita (exe)	34	11,939	Main window, 68 RDLC reports, 24 FFDData and 24 MXFD statutory templates
Servers, plugins, helpers	Remoting server, ASMX web service, plugin manager, common helpers, database migration tool, invoice exchange format	56	8,196	CSLA data portal over HTTP, a small REST endpoint for invoices, an invoice-import adapter interface

Findings that shape the design

Finding	Evidence in the code	Design consequence
Money is binary floating point everywhere	228 DOUBLE money columns and no DECIMAL in the 67-table schema; 3,295 <code>As Double</code> in the business layer; rounding helper <code>CRound</code> called 3,876 times in 219 files	Target stores money as exact decimal. Equivalence is defined after rounding to the cent, and legacy rounding is ported as one tested library function (L3).

Finding	Evidence in the code	Design consequence
The same rounding exists in five copies that disagree	Three VB copies of <code>CRound</code> , a SQLite function and a MySQL stored function. VB compares through a 15-digit Decimal; MySQL compares in DOUBLE.	By emulation, 1.005 rounds to 1.01 in VB and 1.00 in SQL; about 1 in 550 VAT calculations at 21% differ by a cent. This is the first entry in the deviation register and the reason L3 compares to the cent.
No foreign keys, positional column mapping	0 foreign keys and 36 indexes; 132 columns describe their reference only in a free-text description; fetches read <code>dr.Item(0)</code> to <code>dr.Item(21)</code> by position	L2 infers relationships from descriptions and SQL joins, then profiles every database for orphans before migration.
Each tenant's schema state is unknown	No schema version table. The user upgrades by hand through "check and fix" (a diff of an XML schema against the live database); older versions are detected by sniffing columns.	The pre-flight scan reuses the legacy diff to bring each of the 4,200 databases to the latest structure before extract.
Statutory change is copy-paste in code	28 declaration classes across 13 form families, one class per version (SAM has seven). Validity dates are hardcoded, and the version list is hardcoded in the forms layer. FFDData fields are filled by table and row position. 20 hardcoded year checks, such as the 2019 social-insurance reform gross-up of 1.289.	This is the mechanism behind the 10 to 14 weeks. It becomes data in rule packs with effective dates.
Every customer maintains its own tax rates	VAT, Sodra, PSD and income tax rates are typed in per company and copied onto each wage sheet. The non-taxable income formula is a per-company editable string.	Rule packs publish rates centrally; per-company overrides become an audited exception. This removes a class of support tickets.

Finding	Evidence in the code	Design consequence
No automated tests	No test project or test framework anywhere in the 12-project solution	The L3 harness is the first safety net, built before any module is rewritten.
Business logic is mostly UI-free	One <code>MsgBox</code> in the business layer and no SQL in forms. Exceptions: a 6,701-line print mapper, import screens, and the declaration registry in the forms layer.	The business layer can run headless as the L3 reference (A13) and be translated; print and import code is rebuilt.
SQL is centralised but outside the compiler	770 MySQL and 761 SQLite statements in two XML dictionaries that have drifted (24 keys differ); 672 keys used in code; one raw SQL call	L0 parses the dictionary into a query-to-table map, a fast start for the rule inventory and the schema mapping.
A single global company context	<code>GetCurrentCompany()</code> is read 321 times in 74 files; a process-wide cache is used 141 times	That one seam becomes the tenant context in the target. Every cache key must carry the tenant.
Keyboard habits are specific, not shortcut-heavy	Insert and Delete add and remove grid rows; Enter and Tab move across cells; pickers filter as you type; dates accept shorthand ("5" is the 5th of this month, "-5" last month); almost no menu shortcuts	These exact behaviours define the web client's classic key map and parity tests.
Reports bind to positional columns	68 RDLC reports on ReportViewer 8.0 (2005) fed by a generic data set of 345 <code>ColumnN</code> fields; 17 referenced reports are missing from the repository	Reports are rebuilt on typed report models. The missing 17 are requested from the firm in week 1.
Security debt	A plaintext database password in the shipped web configuration; MD5 still allowed for password hashes; frameworks out of mainstream support	Fixed by design in the target: managed identity, Key Vault, modern identity provider. Rotate the exposed credential now.

Finding	Evidence in the code	Design consequence
Built for small companies	The author calls it unsuitable above about 20,000 operations a year	Supports A3: most tenants are small, which favours pooled tenancy and fast nightly migration.

Assumptions

The brief leaves fifteen facts open; each is stated below with the reason it was needed, what it changed in the design, and how it gets confirmed in the first eight weeks. Later sections refer to them by ID.

ID	Assumption	Why it was needed	How it shaped the design	Confirmed by
A1	Target cloud is Microsoft Azure, in two EU regions (primary and disaster recovery).	No cloud is named; the stack is .NET and customers are EU companies under GDPR.	Managed PostgreSQL, Azure Container Apps, Azure OpenAI in the EU Data Zone. Containers and PostgreSQL keep the design portable to AWS or GCP.	CTO decision, week 1
A2	The 3.1M EUR a year covers hosting, 4,200 MySQL databases, backups, remote-access servers and per-fork build pipelines. Salaries are excluded.	Needed a baseline for the 45 percent target.	Baseline run cost is 738 EUR per customer per year, so the target is 406 EUR or less.	Finance cost-centre extract, week 2
A3	The median company database is under 2 GB and the largest 5 percent are under 50 GB.	Data size decides pooled versus dedicated databases and how fast migration can run.	Pooled database by default, dedicated database for the largest tenants; 100 or more tenant migrations a night.	Size scan of all 4,200 databases, week 2

ID	Assumption	Why it was needed	How it shaped the design	Confirmed by
A4	About 9 users per company on average; a large share of users work for accounting bureaus that keep books for many companies.	38,000 users across 4,200 companies implies shared bookkeepers.	One login spans many companies; bureaus migrate as a portfolio.	Login and licence data, week 3
A5	Lithuania is the main jurisdiction (the code targets VMI and Sodra); Latvia and Estonia come later.	The brief says Baltic; the code base is Lithuanian.	Jurisdiction is a dimension of every statutory rule pack; Lithuania ships first.	Customer list by country, week 1
A6	The firm has about 30 engineers, four of whom hold the tribal knowledge; a Persistent team of about 32 joins for 12 months.	The brief gives the 62 percent maintenance share but not headcount.	Sizes the team plan and the value of freed engineering capacity.	HR headcount, week 1
A7	The 310 forks mostly change reports and print forms, custom fields, imports and exports, and validation rules, not core posting logic.	Collapsing forks depends on what they change.	Three tiers of extension points instead of code forks. Forks that do touch posting logic move to later waves.	AI fork-diff analysis, week 6
A8	Lithuania does not certify accounting software; filings must validate against the tax authority's published XML schemas.	Decides whether a certification gate sits on the critical path.	No certification gate; schema validation and golden-file tests are release gates.	Legal review, week 3

ID	Assumption	Why it was needed	How it shaped the design	Confirmed by
A9	Accountants will accept a browser product if keyboard speed matches the desktop client.	Decides whether a desktop wrapper is needed.	Keyboard-first web client with a classic key map and installable app (PWA); parity is measured per workflow.	Usability tests with 20 power users, week 8
A10	"Zero regression in financial correctness" means ledger, VAT and payroll outputs match legacy to the cent on the same inputs. Known legacy defects are fixed only through a signed deviation register.	The goal needs a testable definition.	Drives the L3 differential test design and every migration gate.	Steering committee sign-off, week 2
A11	Statutory turnaround runs from official publication of a change (law, form or XML schema) to its availability in production.	The KPI needs a start and an end.	Statutory rules ship as configuration packs, outside the code release train.	KPI charter, week 2
A12	A support ticket costs 12 EUR fully loaded.	Needed to value the reduction in tickets.	Business case for the support assistant.	Support cost data, week 3
A13	The legacy business objects can run without the WinForms client.	The differential test harness drives legacy code without a UI.	Legacy runs headless in containers as the reference oracle for L3.	Spike in weeks 2 to 4

ID	Assumption	Why it was needed	How it shaped the design	Confirmed by
A14	Hosted AI models may process code and customer data under an EU data-processing agreement, with no training on customer data. The base code is already open source; consent is needed for customer forks.	AI is used across the modernization lifecycle and in the product.	EU-hosted models, prompt logging, PII redaction, and per-customer consent before fork analysis.	DPO and legal, week 2
A15	Accounting records must stay readable for at least 10 years.	Decides when legacy data can be retired.	Legacy databases become read-only archives with SAF-T exports, not deletions.	Legal review, week 3

Target architecture

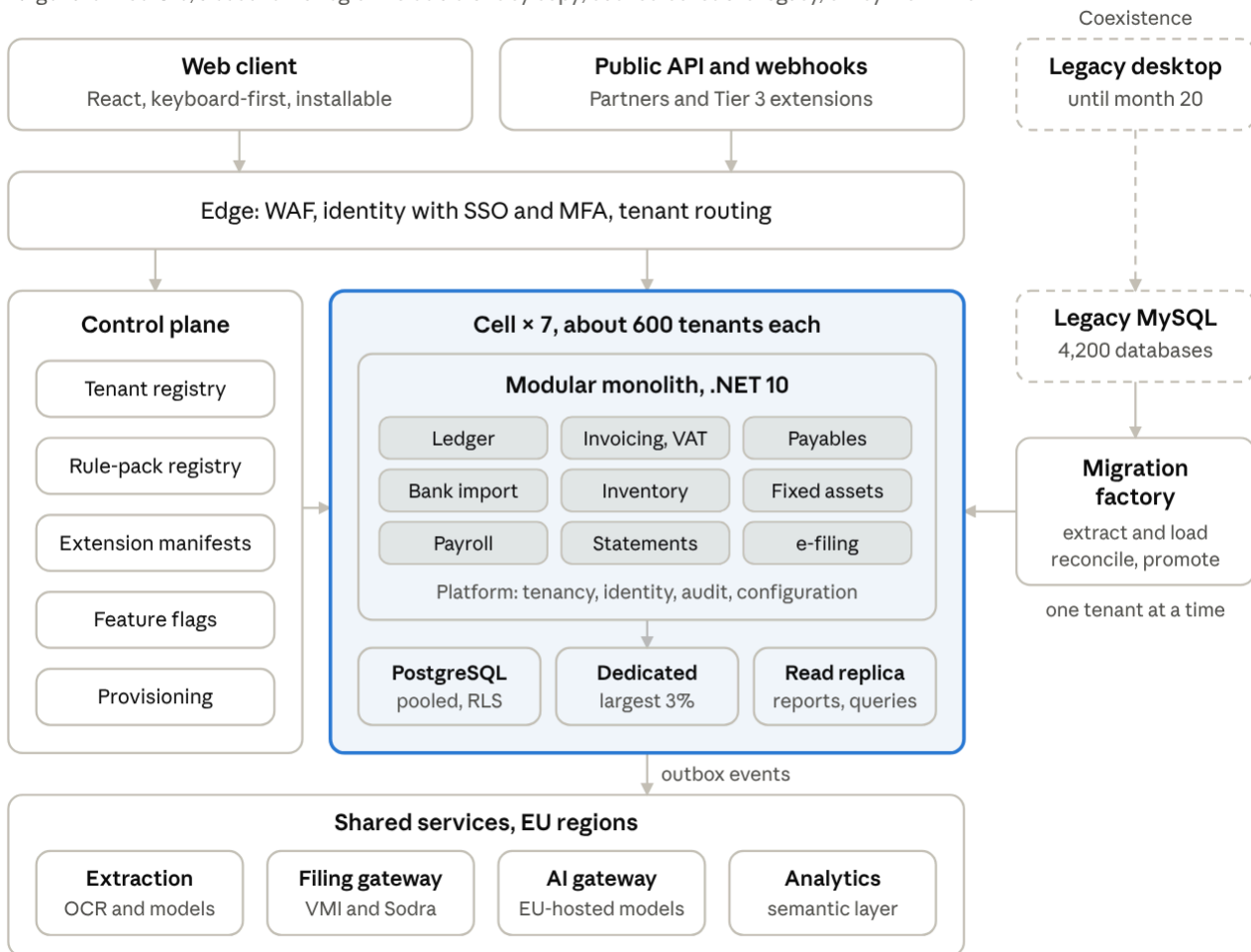
The target is one code base for every customer, built from five parts:

- a modular monolith on .NET 10;
- pooled PostgreSQL with row-level security, deployed in cells of about 600 tenants;
- a keyboard-first web client;
- statutory rules shipped as versioned configuration packs;
- three tiers of extension points that replace all 310 forks.

Each choice is the simplest option that meets the five business goals. The alternatives are weighed in the trade-off section.

Every tenant runs one release inside pooled cells

Target architecture; a second EU region holds a standby copy; dashed boxes are legacy, off by month 20



Target architecture · clients, edge, control plane, cells, shared services, coexistence

Requests pass the edge, where the control plane's tenant registry routes each one to the tenant's cell. Inside a cell, one modular monolith serves every tenant from pooled PostgreSQL, or from a dedicated database for the largest. Legacy keeps serving unmigrated tenants until the migration factory moves them, one tenant at a time.

Tenant isolation model

Decision: pooled database by default, dedicated database by exception, both inside cells.

- **What a tenant is.** One tenant is one set of books, the same unit as one of today's 4,200 databases, so migration maps one database to one tenant. Accounting bureaus are organisations in the identity layer whose users hold memberships in many tenants (A4).
- **Pooled tier.** All tenants in a cell share one PostgreSQL schema. `tenant_id` leads every primary key, foreign key and index. The API sets `app.tenant_id` from the verified token on every connection, and PostgreSQL row-level security policies filter every

table. The application role owns no tables, so it cannot bypass the policies. An EF Core global query filter is a second, independent layer.

- **Dedicated tier.** About 3 percent of tenants, the largest or those with contractual isolation (A3), get their own database in the same cell, with the same schema, code and release. The tenant registry routes each request, and moving between tiers uses the migration tooling.
- **Cells.** A cell is about 600 tenants with its own app instances, database cluster and read replica: 7 cells at full scale. A cell caps the blast radius of a fault, is the unit of canary deployment, and adds capacity in steps instead of per customer.
- **Keys and data lifecycle.** Payroll and personal data columns are encrypted with a per-tenant data key held in Key Vault, so deleting the key crypto-shreds an off-boarded tenant. Each tenant can export its books as SAF-T plus a native archive, and a single tenant can be restored to a point in time without touching its neighbours.
- **Noisy neighbours.** Per-tenant rate limits and statement timeouts apply. Heavy reports run on the read replica or as background jobs.
- **Proof of isolation.** Automated cross-tenant probes run in CI and every hour in production against canary tenants. An external penetration test is a gate before the first wave.

Modular monolith, not microservices

Decision: one deployable with nine enforced business modules; separate services only where the runtime profile differs.

- **Modules.** The nine in scope: Ledger, Invoicing and VAT, Payables, Bank import, Inventory, Fixed assets, Payroll, Statements, and Statutory e-filing, plus a Platform module (tenancy, identity, audit, configuration).
- **Boundaries.** Each module owns its own PostgreSQL schema and exposes a published interface. Architecture tests in CI fail any call or table access across a boundary.
- **Posting.** Posting an invoice touches sales, VAT, inventory and the ledger in one ACID transaction, so financial correctness never depends on distributed consistency.
- **Events.** Other consumers (anomaly detection, analytics, search, webhooks) receive domain events through a transactional outbox, so a module can later be extracted without rewriting its callers.
- **Separate services.** Four workloads run separately because their load differs. They are document extraction, the filing gateway to the tax authority and Sodra, analytics, and the migration factory.

Web client for keyboard-heavy accountants

Decision: React and TypeScript single-page app with a spreadsheet-grade grid, designed keyboard-first and measured against the desktop.

- **Muscle memory kept.** A "classic" key map copies what the desktop does today. Insert and Delete add and remove rows, and Enter and Tab move across cells. Pickers filter as you type, and dates accept shorthand such as "5" or "-5". A "modern" map adds documented shortcuts.
- **Data entry.** Journal, invoice and payroll lines use a grid that behaves like a spreadsheet:
 - type-ahead lookups for accounts, counterparties and items;
 - row insert and duplicate from the keyboard;
 - paste of blocks from Excel;
 - validation inline, not in modal dialogs.
- **Navigation.** A command palette (Ctrl+K) opens any document, report or action. Tabs replace the desktop's multiple windows, and a company switcher serves bureau users.
- **Speed.** Keystroke to screen stays local under 50 ms. The server is called on commit and for lookups, with drafts saved as the user types so nothing is lost on a dropped connection.
- **Parity as a release gate.** The 25 most frequent workflows are timed in keystrokes and seconds on legacy and on web with power users. A module ships to customers only when 90 percent of its workflows are as fast or faster (A9).
- **Standards.** WCAG 2.2 AA, Lithuanian and English, installable as a desktop app (PWA) for users who want a taskbar icon.

Statutory variability as configuration

Decision: statutory rules live in signed, effective-dated rule packs released independently of the application.

Today a rate change, a new declaration version or a new XML schema means a code change, a build of every fork and a twice-yearly release. That is why turnaround is 10 to 14 weeks. In the target, the code is a generic engine and the law lives in data. A rule pack per jurisdiction holds four things:

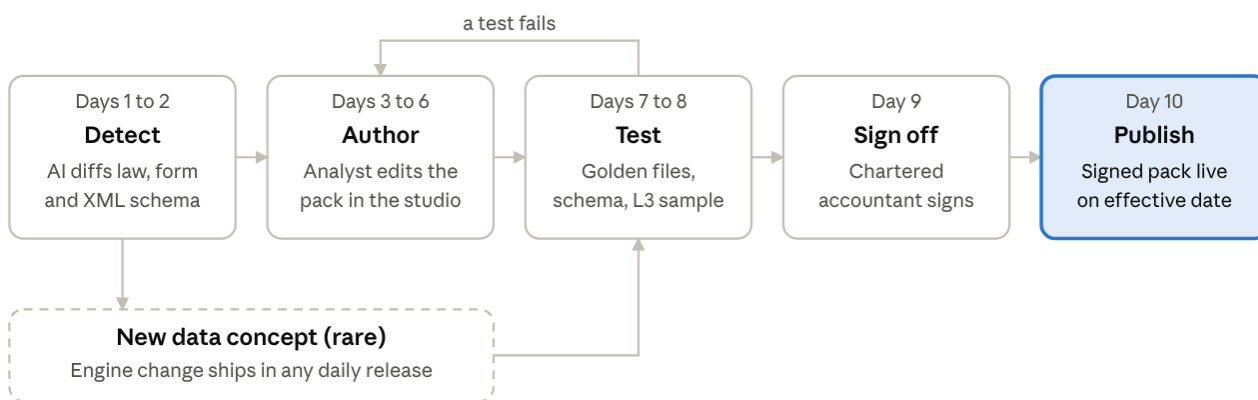
1. **Parameters.** Effective-dated tables: VAT rates, social insurance and income tax rates, ceilings, the non-taxable income formula coefficients, minimum wage.
2. **Calculation rules.** Decision tables and expressions in a deterministic, decimal-only rule language with explicit rounding. The language has no I/O and no loops, so a pack cannot harm the platform.

3. **Filing definitions.** Which ledger and sub-ledger queries fill which form fields, the output format (XML against the authority's schema, or FFData), and cross-field validations.
4. **Test fixtures.** Golden input and output files, including the authority's own published examples and anonymised past filings.

A statutory desk of two analysts and one engineer owns the packs. The engine itself changes only when the law introduces a new concept that needs new data, such as a new mandatory invoice field. That is rare, and it ships in an out-of-cycle release, which the pipeline supports on any day.

A statutory change ships as data in about 10 working days

Worst case 14 working days (20 calendar days), with no code release; legacy takes 10 to 14 weeks



Statutory change pipeline · 5 steps, budget in working days

Most changes are new rates, new form versions or new schema fields, and they travel the top row as data. Only a change that needs new data goes through an engine release. That release can ship on any day, so it meets the same 20-day limit.

From 310 forks to extension points

Decision: every fork is decomposed into configuration, templates and rules, or out-of-process code extensions. Core code is never forked again.

AI fork analysis in L0 diffs each fork against the release it branched from and clusters the changes (A7). Each change is then mapped to the lowest tier that can carry it:

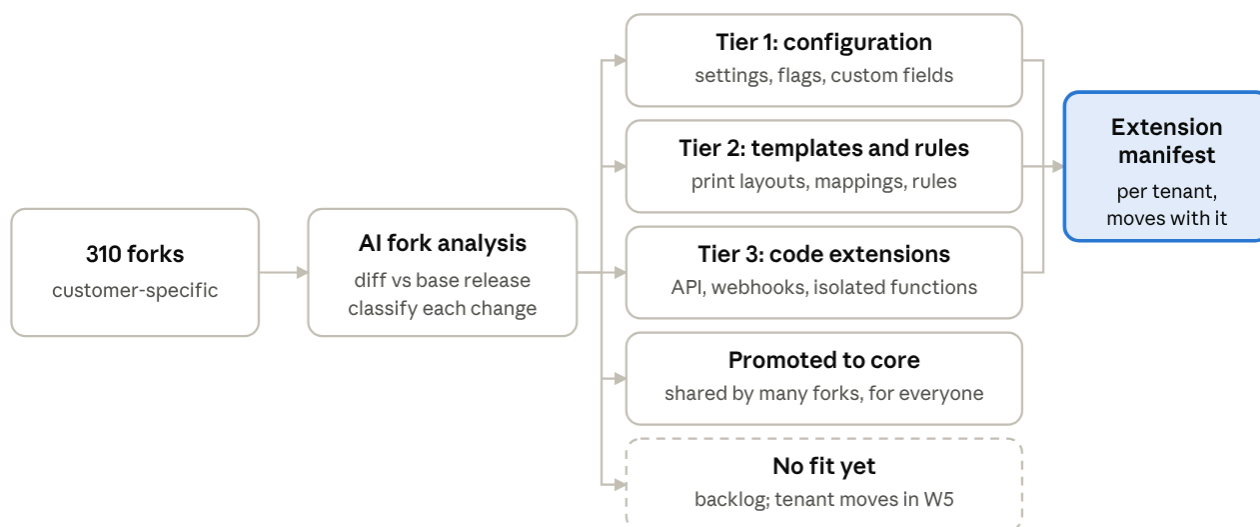
- **Tier 1, configuration.** Settings, feature flags, numbering series, approval thresholds and custom fields stored as typed JSON with a per-tenant schema.
- **Tier 2, templates and rules.** Print and report templates, import and export mappings, and extra validation or approval rules written in the same rule language as the statutory packs.

- **Tier 3, code extensions.** Logic that needs code runs outside the core process: through the public REST API, webhooks and event subscriptions, with a connector SDK, hosted in isolated functions per tenant.
- **Promoted to core.** Changes that many forks share become product features for everyone.

Each tenant migrates with an extension manifest listing its configuration, templates, rules and connectors. A fork whose change fits no tier becomes a product backlog item, and that tenant moves to a later wave rather than keeping a fork.

Every fork change lands in a tier, in the core, or on the backlog

Fork consolidation; no tenant keeps a code fork in the cloud



Fork consolidation · 310 forks into three tiers, core, or backlog

Tiers 1 to 3 together form each tenant's extension manifest, which the migration factory applies at cutover. The legacy code already has a precedent: a working invoice-import adapter interface with one external plugin. Tier 3 generalises it and moves it out of process.

Technology stack

Layer	Choice	Why
Web client	React, TypeScript, AG Grid, installable PWA	Spreadsheet-grade keyboard grid; largest hiring pool
API and modules	.NET 10 (LTS), ASP.NET Core, C#	Long-term support; continuity for a .NET team
Data access	EF Core with Npgsql; Dapper for heavy reads	Global tenant filter as the second isolation layer

Layer	Choice	Why
Database	Azure Database for PostgreSQL Flexible Server, zone-redundant, one read replica per cell	Row-level security, exact numeric, managed high availability
Compute	Azure Container Apps per cell, with event-driven autoscaling for workers; the migration factory runs as Container Apps jobs, with the same .NET pipeline code in rehearsal and at cutover	Containers without operating Kubernetes ourselves
Messaging	Azure Service Bus with a transactional outbox	Domain events that are never lost or duplicated
Edge and identity	Azure Front Door with WAF; Microsoft Entra External ID (OIDC, MFA)	Single sign-on across a bureau's companies
Keys and secrets	Azure Key Vault, managed identities, per-tenant data keys	No credentials in configuration files
Documents	Azure Blob Storage with immutability policies	Filed returns and source documents kept write-once
Rule packs	Decision tables and a small in-house, decimal-only expression language; packs signed and stored in the control plane	Analysts can author it; it cannot do I/O
Reporting	Typed report models rendered to PDF and Excel	Replaces RDLC and the 345 positional columns
AI services	Azure AI Document Intelligence; Azure OpenAI (EU Data Zone) behind a model gateway; pgvector for retrieval	EU residency; models can be swapped
Analytics	Read replica plus a semantic layer of finance-owned metric definitions	Governed numbers for NL reporting
Observability	OpenTelemetry to Azure Monitor and Grafana	Views per tenant and per cell
Delivery	GitHub Actions, Terraform, feature flags, canary cell first	Fortnightly releases; daily when needed

Layer	Choice	Why
Modernisation tooling	Roslyn analysers for VB.NET and C#; code models through the gateway; FsCheck property tests; legacy harness in Windows containers on an AKS Windows node pool	Every layer of L0 to L3 runs as automated pipelines

Non-functional targets

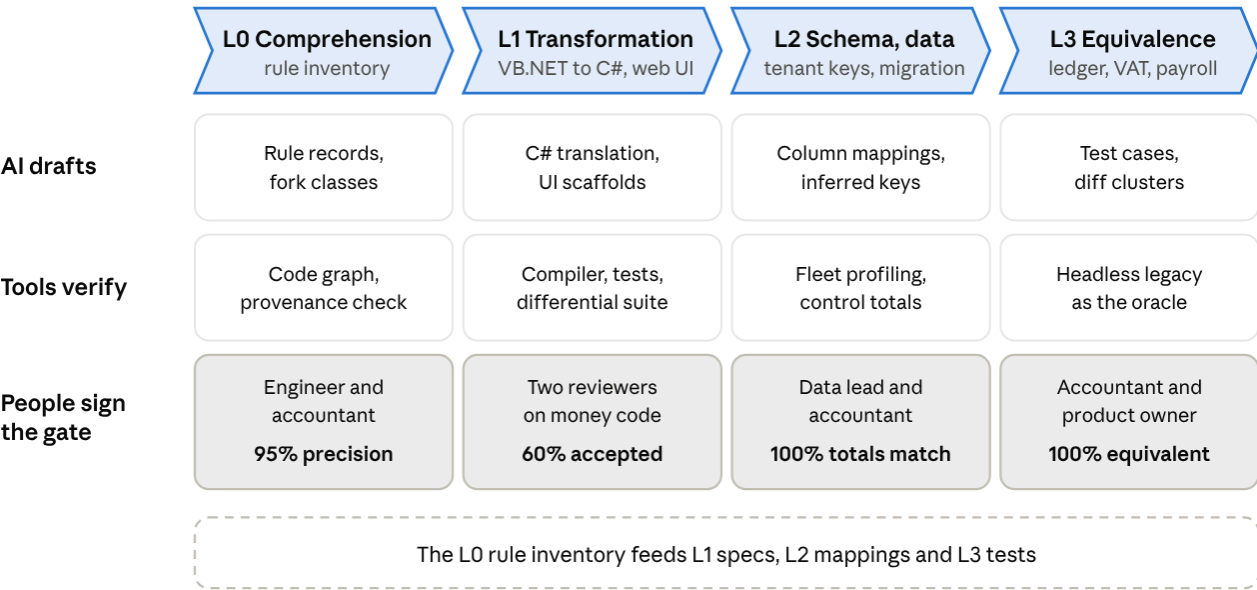
Area	Target
Availability	99.9% a month; 99.95% from the 15th to the 25th, when filings fall due
Recovery inside a region	Zone-redundant databases: no data loss, service back within 1 hour
Disaster recovery	Standby copy in a second EU region: at most 15 minutes of data lost, service back within 4 hours
Peak load	Each cell sized for 3 times its average load during filing week
Speed	Keystroke to screen under 50 ms; API responses under 300 ms at the 95th percentile; month-end reports under 10 seconds
Security	Hourly isolation probes; yearly external penetration test; controls aligned with ISO 27001
Data residency	All customer data and AI processing stay in EU regions

Layered AI in the modernisation lifecycle

At each of four layers, AI does the reading, drafting and comparing. Deterministic tools then check everything it produces, and a named person signs a gate with a numeric threshold before work moves on. AI never moves financial data at run time and never approves its own output.

AI drafts, tools verify, people sign: the same pattern at every layer

Modernisation lifecycle; a layer's output moves on only after its gate passes



Layered AI · four layers, three roles, one gate each

Read across a row to see one role through the lifecycle, or down a column to see one layer's whole control loop. The bottom band is why L0 comes first: every later layer is checked against the inventory it produces.

Checkpoints and thresholds at a glance

Layer	AI drafts	Tools verify	Person who signs	Threshold to pass	If it fails
L0 Comprehension	Rule records, fork classification, ticket clusters	Parser-built code graph; every cited source line must exist and contain the named symbols	Legacy engineer and chartered accountant per module	95% precision on review; 100% of classes, data-portal methods and SQL keys mapped; zero unmapped items in posting, VAT or payroll code	Module re-extracted with reviewer corrections as examples; it does not enter L1

Layer	AI drafts	Tools verify	Person who signs	Threshold to pass	If it fails
L1 Transformation	C# translation of business logic; UI scaffolds from screen specs	Compiler, static analysis, architecture tests, rule-derived unit tests, module differential suite	Two reviewers on posting, VAT and payroll changes; one elsewhere	Every pull request green before review; 60% or more accepted per module	Under 40% accepted for two sprints: the module switches to engineer-led rebuild
L2 Schema and migration	Column mappings, inferred foreign keys, data-quality rules	Profiling across all 4,200 databases; idempotent pipeline; control totals	Data lead and accountant approve each module's mapping	100% of control totals match to the cent for every tenant	Tenant not promoted; legacy stays primary; exception queue
L3 Equivalence	Generated test cases, diff clustering and root-cause hints	Headless legacy as oracle; canonical comparison of every output	Chartered accountant and product owner sign each deviation	100% equivalence on ledger, VAT and payroll; zero unsigned deviations	Release blocked for that module

L0: code comprehension and the business-rule inventory

Goal: a validated, queryable inventory of every rule the system enforces, linked to the exact code that enforces it.

- 1. Parse first, then read.** A Roslyn parser for VB.NET builds a code graph of 377 CSLA classes, 425 data-portal methods and the calls between them. It also lists 797 rule registrations, 328 custom rule methods, 343 role checks and 1,500+ field attributes that carry validation and rounding metadata. The 770-statement SQL dictionary is parsed into a map from key to statement to tables and columns. This step uses no AI and gives the AI a complete map to fill in.
- 2. Extract rules.** For each business object and rule site, an AI agent writes a rule record with these fields:
 - ID and module;
 - kind: validation, calculation, posting, statutory or authorisation;
 - a plain-language statement and a formal expression;
 - inputs, outputs and effective dates;
 - the source lines and SQL keys it comes from;
 - a confidence score.

3. **Extract twice.** Two independent passes with different prompts and models. Disagreements go to a reviewer queue. A mechanical provenance check rejects any record whose cited lines do not contain its named symbols.
4. **Analyse forks.** Each fork is diffed against the release it branched from. Every change is classified to an extension tier, a core feature, or obsolete (A7).
5. **Mine tickets and the manual.** Twelve months of tickets are clustered and mapped to rules and modules, exposing defect hot spots and behaviours nobody wrote down. The user manual is mined for documented behaviour.

The output is a database, not a document. It seeds L1 specifications, L3 tests (at least one per rule) and the first statutory rule packs. We expect 2,000 to 3,000 rule records.

Review is split by risk. The four legacy engineers and a chartered accountant review 100 percent of posting, VAT, payroll and statutory records, plus a 20 percent stratified sample of the rest. Their time is protected: one engineer from each module pod takes over their maintenance work from week 2.

L1: VB.NET and WinForms transformation

Rule: translate the kernel, rebuild the shell.

- **Translate** calculation code that is dense, correctness-critical and stable: posting, VAT calculation, costing, depreciation, payroll calculation, currency revaluation. AI translates VB.NET to C# and reshapes CSLA objects into domain entities and services. The structure stays close to legacy, so any differential failure points to one method.
- **Rebuild** from the rule inventory wherever legacy code is a poor guide:
 - forms, from AI-extracted screen specs (fields, validations, tab order, key bindings) that generate React scaffolds;
 - data access written for positional SQL;
 - declarations, which become rule packs;
 - reports, which become typed report models;
 - bank formats, which become connectors.

Module	Legacy footprint (files, lines)	Approach	Reason
Ledger	42, 17,035	Translate posting, closing and balance transfer; rebuild screens	Core correctness; compact and stable

Module	Legacy footprint (files, lines)	Approach	Reason
Invoicing and VAT	38, 24,704	Translate invoice and VAT calculation and the 12 invoice-to-goods and invoice-to-asset adapters; VAT return mapping to a rule pack	Posting logic is dense; declaration mapping is volatile
Payables	15, 11,475	Translate posting; rebuild intake around AI extraction	The workflow changes most for users
Bank import	22, 29,408 (18,394 generated)	Rebuild: ISO 20022 camt.052 and camt.053 and pain.001 on standard libraries, LITAS-ESIS and custom formats as connectors; translate payment posting rules	Most of the code is generated parsers
Inventory	63, 38,208	Translate FIFO, LIFO and average costing and production operations; rebuild screens	Costing is the subtlest calculation in the system
Fixed assets	33, 29,967	Translate depreciation and value-change operations	Long-lived calculations must match for years of history
Payroll	31, 19,660	Translate the calculation kernel; move rates, formulas and the 20 hardcoded year rules into the payroll rule pack; rebuild timesheets	Kernel is exact; parameters are volatile
Statements	100, 26,031	Rebuild on typed report models; translate the statement-structure logic	Reports bound to 345 positional columns cannot be ported
Statutory e-filing	32, 31,162	Rebuild as rule packs; the legacy classes are the specification and the test oracle	Copy-paste versioning is the root cause of slow change

Module	Legacy footprint (files, lines)	Approach	Reason
Shared business code	157, 29,705	Translate shared validation and field attributes into the domain layer; rebuild lookups and settings on the new schema	Used by every module
Localised resources	4, 47,233 (generated)	Regenerate as resource files in Lithuanian and English	Generated strings, not logic
Desktop UI	414, 180,484	Rebuild as the web client	No useful automatic WinForms-to-web path

The first eleven rows cover the whole business layer, about 304,700 lines.

Each AI translation is one pull request per aggregate, with the rule records it implements attached. It reaches a reviewer only after the compiler, static analysis, architecture tests, rule-derived unit tests and the module's differential suite are all green.

L2: schema modernisation, tenant keys, migration and reconciliation

1. **Target schema.** One PostgreSQL schema per module.

- `tenant_id` leads every key and index, and row-level security applies everywhere.
- Foreign keys are real.
- Money is exact `numeric`. Each column takes the precision the legacy field attributes declare: 2 places for amounts, 4 to 6 for unit values and rates.
- Keys are UUIDv7, with the legacy ID kept for traceability.

2. **Mapping specs.** AI drafts the column-level mapping from the inventory, the schema XML and the field attributes: source, target, transform, precision and null handling. Engineers approve it. The spec compiles into deterministic, idempotent pipeline code, with no model in the data path.

3. **Relationships.** AI proposes foreign keys from the 132 description-only references and from the joins in the 770 SQL statements. Profiling across all 4,200 databases confirms them and lists orphan rows. Accountants approve the repair rule for each orphan class, or the rows are quarantined and reported to the customer.

4. **Per-tenant pipeline.** Each tenant runs through six steps:

- pre-flight: legacy schema diff to the latest version, plus data-quality profiling;
- read-only extract;

- transform;
- load to a staging partition;
- reconcile;
- promote, by switching the tenant registry.

5. Reconciliation, to the cent or the unit. The control totals are:

- trial balance per account and period;
- VAT by rate, period and return line;
- open receivables and payables per counterparty;
- stock quantity and value per item and warehouse;
- asset cost, accumulated depreciation and net book value per asset;
- payroll year to date per employee and pay element;
- row counts and content hashes per table.

The balance sheet, income statement, VAT return and i.SAF are also produced on both sides and compared canonically.

6. Fleet rehearsal. From month 4, every one of the 4,200 databases is migrated into staging every week. Data-quality problems surface months before a tenant's wave, not on its cutover night. Staging sits inside the production security boundary, with the same encryption and access controls, and is wiped after every run.

Migrated history is never recalculated. It moves exactly as legacy computed it and is proven by reconciliation, while L3 governs every calculation from cutover onward.

L3: differential testing for financial equivalence

Set-up. The legacy business layer runs headless in Windows containers (an AKS Windows node pool) against MySQL copies and is the reference oracle (A13). The new platform receives the same inputs, and the outputs are compared after canonicalisation:

- ledger lines: account, side, amount to the cent, date, document;
- VAT per rate, per return line and per invoice in i.SAF;
- payroll per employee per month: gross, each tax and contribution, net, employer cost;
- every line of every statement.

Four corpora:

1. **Replayed history.** 300 representative tenants, selected by module mix and complexity. Their source documents are re-posted in the new platform, and the results are compared with what legacy stored.
2. **Generated cases.** Property-based tests target the edges:
 - half-cent rounding boundaries;

- multiple currencies;
- reversals and period closes;
- reverse-charge VAT and partial payments;
- FIFO layers with returns;
- sick leave, holiday pay, non-taxable income thresholds and contribution ceilings.

About one million cases per module run every night.

3. **Statutory golden files.** The authorities' published examples and past filings, validated against the official XML schemas and compared canonically.
4. **Rule-derived cases.** At least one test per inventory record, generated by AI. Reviewers check them for posting, VAT and payroll.

Parallel runs. Before payroll customers move (wave W2), the new payroll engine computes two real monthly payrolls in shadow for every W2 tenant, and both must match. Every later payroll tenant runs one matching shadow payroll before its own cutover (gate G0).

Triage. AI clusters the differences and proposes a cause: rounding, ordering, a legacy defect or a new defect. People decide. Each difference ends one of two ways: the new code is fixed, or the difference is recorded as an approved deviation, signed by the chartered accountant and the product owner. The first deviation is already known: the five rounding copies that disagree on half-cent values (see current-state assessment).

Tolerance policy. Each output is compared at the precision its legacy field declares: 2 places for amounts, 4 to 6 for unit values and rates. Moving from floating point to exact decimals produces three kinds of difference, each handled once:

- **Representation noise** below the declared precision is not a difference, because it disappears at the comparison precision.
- **Boundary cases**, where floating point tips a half-cent the other way, are approved once as a class by the chartered accountant and then sampled weekly.
- **Behaviour differences**, where the logic itself differs, are fixed or signed one by one.

AI in the product

Four AI features ship with the cloud product. Each one drafts, flags or explains. Only bounded, reversible and audited actions run without a person, and nothing AI-driven ever files a return, pays a salary or changes a closed period.

Every feature is assigned one of three autonomy levels:

- **Inform.** Flags or explains; changes nothing.
- **Propose.** Prepares a document or match that a person approves with one action.

- **Act within bounds.** Executes a reversible action automatically, inside limits the tenant sets, logged and undoable.

Invoice and bank statement extraction

1. **Intake.** Purchase invoices arrive by a per-tenant e-mail inbox, upload, phone photo or structured e-invoice.
2. **Structured e-invoices** are parsed deterministically. No AI is needed.
3. **PDFs and images** go through OCR and layout extraction. A language model then normalises the lines and proposes the supplier, GL account, VAT code and cost centre, using the tenant's own most similar past invoices as examples.
4. **Deterministic checks** run before anything is shown: VAT arithmetic, supplier VAT number validity (EU VIES), duplicate detection, and open period.

Bank statements in structured formats are parsed deterministically, and AI reads only PDF statements. Exact payment matches (amount and reference) are automatic and rule-based. Fuzzy matches, such as partial payments, one payment for several invoices or name variants, are ranked by a model and proposed for one-click approval.

Ledger anomaly detection

1. **Deterministic checks first:** duplicate supplier invoice numbers, a VAT code that does not fit the counterparty's country, entries dated into closed periods, negative stock.
2. **Statistical models second:** per-tenant baselines over amount, account pair, counterparty, user and timing. Cross-tenant priors use de-identified aggregate features only; no tenant's records leave its partition.
3. **Explanation:** a language model writes the plain-language reason and cites the entries.

Flags land in a review queue and in the pre-filing checklist. A high-severity flag on a VAT period must be acknowledged, with a reason, before filing. The person decides, not the model.

Natural-language reporting with guardrails

The model never writes SQL and never produces a number.

1. **Translate.** It turns a question such as "revenue by customer, Q3 against last year" into a structured query. The query runs against a semantic layer: a catalogue of metrics such as revenue, gross margin and VAT payable, each defined and owned by finance.
2. **Validate.** The query is checked against the catalogue and the user's role. Payroll metrics stay hidden without payroll rights.
3. **Execute.** A deterministic engine runs it on the read replica under the tenant's row-level security.

4. **Answer.** The reply shows the table or chart, the metric definitions, filters and period.

Any narrative inserts numbers from the result by template, never from the model.

Ambiguous questions get a clarifying question back. Row and cost limits apply. Each release must answer a 500-question evaluation set with at least 95 percent exact results.

Grounded support assistant

- **Sources.** It answers only from the user manual, release notes, rule-pack notes, knowledge-base articles and de-identified resolved tickets. Every answer cites its sources, and it says it does not know when retrieval is weak.
- **Tenant context.** With the user's permission it reads their configuration and recent errors through read-only tools. For example, it can explain which entries feed a given VAT return line, and why.
- **Escalation.** It hands over to a human agent with a summary, so the customer is not asked again.
- **Target.** It resolves 30 percent of contacts without an agent by month 18: about 1,950 tickets a month, worth 0.28M EUR a year at A12. Fewer fork defects and on-time statutory changes remove another 10 percent.

Autonomy matrix

Feature	May act alone	Needs a person	Never	Gate before launch	Watched in production
Invoice extraction	Post a purchase invoice when all of these hold: the tenant opted in; the supplier has been seen 5+ times with the same coding; all checks pass; calibrated confidence is 0.98+; the amount is under the tenant limit (default 1,000 EUR); the period is open. Each is reversible in one click.	Everything else: new suppliers, low confidence, over limit.	Post into a closed period; create a supplier's bank details.	97% field accuracy on a gold set	Weekly sample of auto-posts; error rate under 0.5%

Feature	May act alone	Needs a person	Never	Gate before launch	Watched in production
Bank matching	Exact amount-and-reference matches (rule-based).	Fuzzy and split matches.	Initiate payments.	99% precision on exact matches	Unmatch rate
Anomaly detection	Raise flags and explanations.	Acknowledge a high-severity flag before filing.	Change, reverse or block entries.	60% flag precision on labelled history	Flags per tenant (median 5 or fewer a month); feedback
NL reporting	Run read-only queries in the user's own permissions.	Save or share a report.	Write data; produce numbers outside the engine.	95% exact results on 500 questions	Wrong-answer reports; refusals
Support assistant	Answer with citations; open the right screen.	Any change, offered as a button the user presses.	Change data or settings.	90% grounded answers on a gold set	Resolution rate; escalation rate; customer satisfaction

Governance

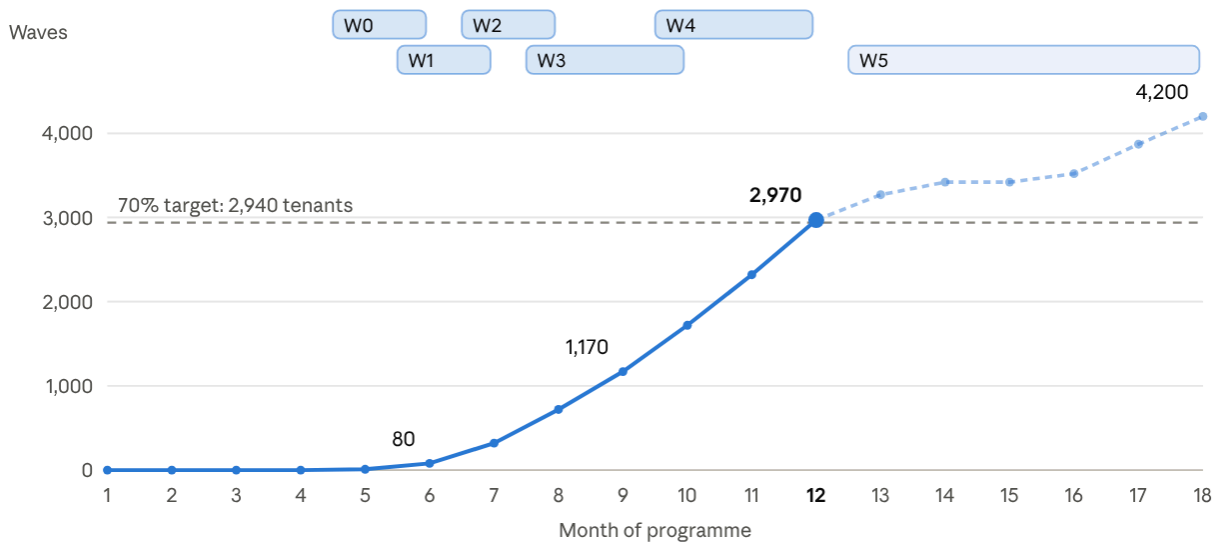
- **Regulation.** All four features are limited- or minimal-risk under the EU AI Act. The assistant and reporting carry transparency notices. Payroll is calculated by deterministic rule packs, not by AI.
- **Models.** EU-hosted models sit behind a model gateway, with Azure OpenAI in the EU Data Zone as default and swappable. There is no training on customer data, and personal data is redacted before support prompts (A14).
- **Audit.** Every AI action records the model and version, its inputs, output, confidence and approver. These records stay with the ledger for the statutory retention period (A15); raw prompts are kept 90 days.
- **Kill switch.** Each feature can be turned off per tenant and globally in seconds.

Migration plan

2,970 customers (70.7 percent) move in five waves between months 5 and 12; the remaining 1,230 follow by month 18, and legacy is switched off at month 20. Each tenant cuts over in one night. Legacy stays read-only behind it, with a tested way back for up to 35 days.

2,970 tenants are live by month 12, just over the 70% target

Planned cumulative tenants migrated by month; wave W5 moves the last 1,230 by month 18



Wave plan in this document · cumulative tenants by month, months 1 to 18

The curve is steepest from month 9, after every module has passed L3 and the factory has run the whole fleet in rehearsal for five months.

Wave sequencing

Wave	Months	Tenants	Cumulative	Who moves	Modules that must have passed L3
W0 Design partners	5 to 6	20	20	The firm's own books and 19 volunteer customers	Ledger, Invoicing and VAT, Payables, Bank import, Statutory e-filing for VAT and i.SAF
W1 Simple	6 to 7	150	170	No payroll, no inventory, no fork, under 1 GB	Same as W0
W2 With payroll	7 to 8	400	570	Adds payroll, after two months of parallel payroll runs that match	Plus Payroll
W3 Bureaus and mid-size	8 to 10	1,000	1,570	Inventory and fixed assets; each bureau's portfolio moves together	Plus Inventory, Fixed Assets, Statements
W4 Forked and larger	10 to 12	1,400	2,970	Tenants whose forks map fully to extension manifests; dedicated-database tenants	All nine
W5 Long tail	13 to 18	1,230	4,200	Forks that needed new product features, unusual integrations, late adopters	All nine

Four rules set the order inside the waves:

1. **Complexity score.** A pre-flight scan scores every tenant on modules used, data size, fork changes, integrations and data-quality findings. The lowest scores go first.
2. **Module readiness.** A tenant moves only when every module it uses has passed the L3 gate.
3. **Statutory calendar.** No cutovers from the 10th to the 25th of a month, when Sodra, income tax, i.SAF and VAT returns fall due. No cutovers from 15 December to 15 February, around year-end. With a November 2026 start, waves W0 to W4 run March to October 2027 and avoid year-end. W5 pauses for the year-end freeze, which is the flat stretch in months 14 and 15 of the chart.
4. **Bureau portfolios.** A bureau's companies move in the same wave, so its accountants work in two systems for weeks, not months.

The migration factory handles 100 tenants a night. The calendar leaves about 10 cutover nights a month, from the 26th to the 9th, so capacity is about 1,000 a month against a planned peak of 650. The headroom absorbs re-runs.

Coexistence model

- **One system of record.** At any moment a tenant is written either in legacy or in the cloud, never both, so there is no two-way sync of financial data.
- **One front door.** A single sign-on portal sends each user to the right system for each company. Bureau users see one company list, with a badge showing where each company lives.
- **Freezes.** Forks are frozen at month 3: no new customisations, and new requests go to the extension backlog. Legacy features are frozen at month 6. From then on a ring-fenced legacy team of two ships statutory updates only.
- **Integrations.** At cutover, a tenant's bank feeds, e-shop connectors and API keys switch to the cloud endpoints from its extension manifest.

Until they move, legacy customers keep today's 10-to-14-week statutory turnaround, because the legacy team still ships each change across every fork. This is the main reason the plan does not let the long tail drift past month 18.

Customer change management

- **Training.** Each wave gets in-app guided tours of the 25 most frequent workflows and a 45-minute live session per role. A one-page sheet maps every desktop keyboard habit to the web client.
- **Champions.** Each bureau names a champion who moves in W0 or W1 and then supports its own staff.

- **Communication.** Customers hear from us at T-14, T-7, T-1 and at go-live, and get a named contact for hypercare.
- **Pricing.** Nobody pays more to move during the programme. Early movers keep their current price for 12 months.

Cutover runbook (per tenant)

When	Step	Run by	Exit criterion
T-14 days	Pre-flight scan: schema version, data quality, fork manifest, integrations. Customer is notified and booked into training.	Factory (automatic)	Gate G0 passed
T-7 days	Full rehearsal migration into staging; reconciliation of all control totals; round trip back to legacy format.	Factory (automatic)	Gate G1 passed
T-7 to T-2	Customer previews the cloud copy and confirms opening balances and key reports.	Customer, customer success	Gate G2 passed
T0, 20:00	Legacy database set read-only for this tenant; final extract.	Factory	Read-only confirmed
T0, 20:15	Transform and load into the cell; extension manifest applied.	Factory	Load complete
T0, about 21:00	Final reconciliation, smoke tests, isolation probe; tenant activated and portal route switched; users notified.	Factory, on-call engineer	Gate G3 passed
T+1 to T+35	Hypercare: priority support queue, daily anomaly scan, ticket rate watched.	Customer success, module pod	Gate G4 passed
After G4, by T+60	Legacy database exported to the encrypted archive and removed from hosting.	Platform	Archive verified

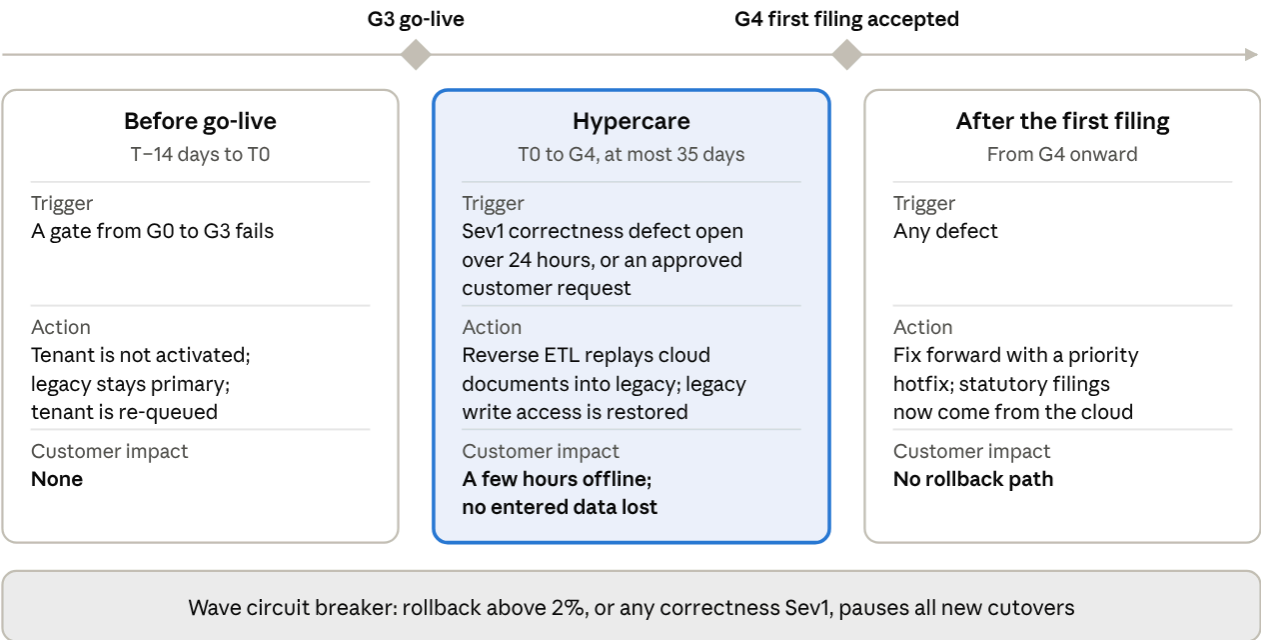
Validation gates

- **G0 Eligibility.** Every module the tenant uses has passed L3, its extension manifest is complete, data-quality findings are fixed, its schema is on the latest legacy version, and a payroll tenant has run one matching shadow payroll.
- **G1 Rehearsal.** Every control total matches to the cent (see L2), report diffs are clean, and the round trip back to legacy is lossless.
- **G2 Acceptance.** The customer or their bureau confirms in the preview. Low-risk W1 tenants may accept silently after five days, with the reconciliation report sent to them.
- **G3 Go-live.** The final reconciliation matches 100 percent, smoke tests pass, and the isolation probe is clean.
- **G4 Hypercare exit.** The tenant's first statutory filings from the cloud are accepted, no Sev1 or Sev2 defect is open, and its ticket rate is at or below its legacy baseline. For VAT payers the first filings are the VAT return and i.SAF. A tenant with no filing due passes G4 at day 35 on the other two criteria.
- **Wave gate.** Waves overlap, so each is judged on its first cohort: the first 20 percent of its tenants, live for at least 30 days. That cohort must show rollback under 2 percent, zero correctness Sev1 defects, and tickets at or below the legacy baseline. The next wave's modules must also pass L3 at 100 percent. The steering committee then signs go or no-go.

Rollback

Rollback is free before go-live and closes at the first filing

Per-tenant rollback paths along the cutover; phases not to scale



Rollback paths · three phases of a tenant cutover

The reverse path is real only because it is exercised. Every G1 rehearsal round-trips the tenant from legacy to the cloud and back, and must come back lossless. Rollback closes at G4: once the authority has accepted a filing produced in the cloud, the cloud is the legal record, and defects are fixed forward.

Legacy end of life

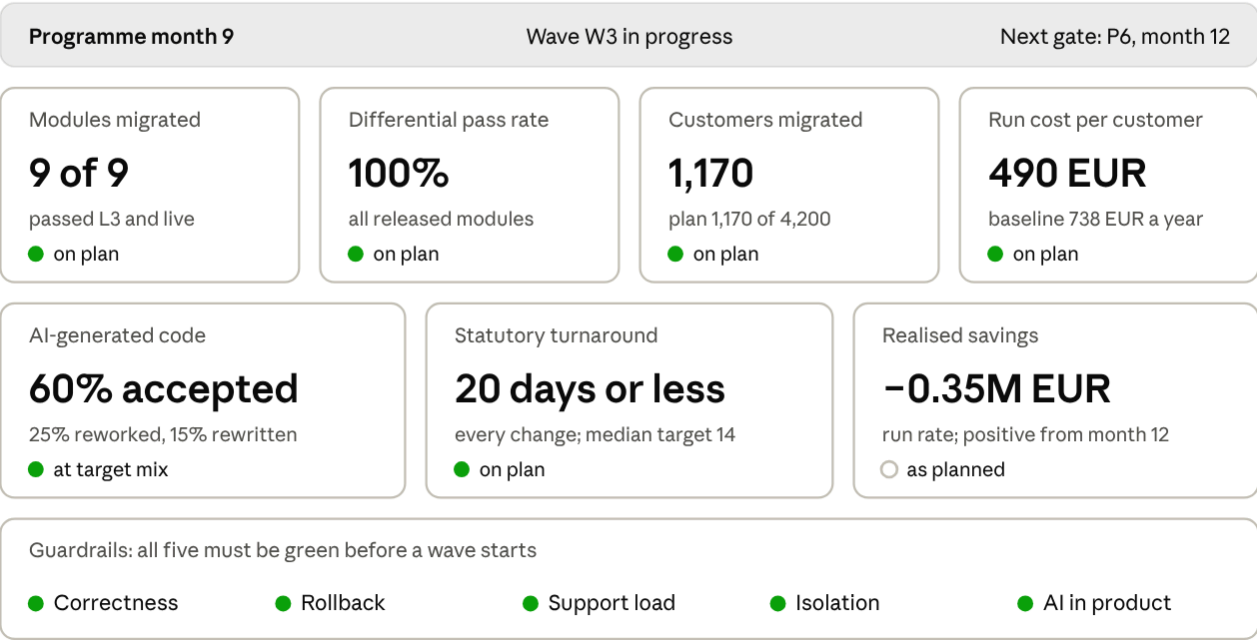
Month	Milestone
3	Fork freeze: no new customisations in legacy
6	Legacy feature freeze: statutory updates only, by a ring-fenced team of two
9	End of life announced: support ends at month 18; migration is free, and early movers keep their current price for 12 months
18	Last legacy statutory update; standard support ends
20	Legacy hosting switched off. Every legacy database is kept as an encrypted, read-only archive for at least 10 years (A15), with a SAF-T export and an archive viewer.

KPI dashboard

Seven programme KPIs and five guardrails are published weekly from live data. The steering committee reviews them monthly against the targets below. Every KPI is computed by a query, not typed into a slide, so the dashboard and the business case cannot drift apart.

One page tells the steering committee whether to start the next wave

Mock-up showing month 9 plan values; every tile is a live query



The top row answers "are we on plan", the middle row "is the method working", and the bottom strip "is it safe to continue". A red guardrail overrides any amount of green above it.

Programme KPIs

KPI	Definition	Source	Month 6	Month 9	Month 12
Modules migrated	Modules out of 9 that passed the L3 gate and run for customers	Release registry	5	9	9, with all extension tiers
Differential test pass rate	Equivalent cases divided by all cases, per module, across generated, replayed and statutory corpora; open deviations shown beside it	L3 harness	100% on W0 modules	100% on all released modules	100%; zero unsigned deviations

KPI	Definition	Source	Month 6	Month 9	Month 12
AI-generated code accepted versus reworked	Share of AI-generated changes merged with under 10% of lines changed in review (accepted), 10 to 50% (reworked), over 50% or discarded (rewritten), by module and artefact type	Pull-request metadata	Baseline set on Ledger	60% accepted or more	60% accepted, 15% rewritten or fewer
Customers migrated against plan	Tenants live in the cloud and past gate G3, cumulative, against the wave plan	Tenant registry	80	1,170	2,970 (70.7%)
Run cost per customer	Cloud run cost over tenants live in the cloud, annualised; blended cost across all 4,200 customers shown alongside (baseline 738 EUR)	Cloud billing by cell, tenant registry	Not meaningful under 100 tenants	490 EUR or less	406 EUR or less (model: 330 EUR)
Statutory turnaround	Calendar days from official publication of a change to its rule pack in production; median and worst case	Rule-pack registry	First pack change within 20 days	Every change within 20 days	Every change within 20 days; median 14
Realised savings	Baseline 3.1M EUR minus actual cloud plus legacy run cost, plus avoided support cost, as an annual run rate	Finance, cloud billing, support desk	–0.38M EUR run rate	–0.35M EUR run rate	+0.19M EUR run rate; 2.2M EUR from month 21

Guardrails

A breach of any guardrail pauses all new cutovers automatically:

- **Correctness.** Zero financial-correctness incidents in production (ledger, VAT, payroll).

- **Rollback.** Under 2 percent of tenants in a wave.
- **Support load.** Tickets per migrated tenant per month at or below the legacy baseline of 1.55.
- **Isolation.** 100 percent of hourly cross-tenant isolation probes pass.
- **AI in the product.** Auto-post error rate under 0.5 percent; NL reporting exact-result rate at or above 95 percent.

Realised savings stay negative until about month 12, because legacy hosting falls only as databases are archived after hypercare. The business impact section shows the curve.

Trade-off analysis

Every major decision below favours financial correctness and a single code base over speed of the first release. The cost of each choice is stated next to it, because each one has a cost.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
Modernisation strategy	AI-assisted re-architecture: translate calculation kernels, rebuild the shell, migrate tenant by tenant	Rehost the desktop app on cloud virtual desktops; manual rewrite; replace with another vendor's SaaS	Rehosting keeps 4,200 databases, 310 forks and the statutory delay, so cost barely moves. A manual rewrite of 522,000 untested lines does not fit in 12 months and loses unwritten rules. Replacing the product ends the business.	The hardest path up front; it depends on the L0 to L3 safety net working.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
Tenant isolation	Pooled PostgreSQL with row-level security, in cells, with a dedicated tier	One database per tenant on a managed service; one schema per tenant	Packing per-tenant databases onto shared servers or elastic pools would cut hardware cost too. But every release would still run 4,200 schema migrations, and per-database connections, caches and backups cap how many fit on a server. Schema per tenant has the same upgrade problem inside one catalogue.	Small tenants share physical storage, so isolation must be proven continuously (probes, penetration test, two filter layers).
Architecture style	Modular monolith with enforced boundaries	Microservices per module	Posting crosses four modules and must be one transaction. A team taking over from four key engineers cannot also run a distributed system.	No independent scaling per module; this is mitigated by cells and by outbox events that allow later extraction.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
Database engine	PostgreSQL	MySQL 8 (current); SQL Server	MySQL has no row-level security, so isolation would rest on application code alone. SQL Server has it but adds licence cost to every cell.	A dialect change; affordable because data access is rebuilt for tenant keys anyway, and L3 catches behaviour drift.
Language	C# on .NET 10	Keep VB.NET on .NET 10	VB.NET runs on modern .NET, but the language is frozen, web tooling targets C#, and C# developers are far easier to hire.	Translation effort, absorbed by AI translation of the business layer.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
Web client	React, TypeScript and a spreadsheet-grade grid	Blazor; stream the WinForms app to a browser	React has the strongest grid and keyboard ecosystem. Blazor Server holds a live connection per user, and every keystroke crosses the network. Blazor WebAssembly avoids that but has a heavier download and fewer grid options. Streaming keeps the desktop architecture.	Two languages in the stack.
Statutory variability	Signed, effective-dated rule packs	Code changes on a faster release train; a commercial rules engine	Continuous delivery alone gets to about 4 to 6 weeks and still needs engineers for each rate change. A commercial engine adds licence cost and a second vendor in the payroll path.	We build and own a small rule language and its authoring and testing tools, and its expressiveness is deliberately limited.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
Fork consolidation	Three extension tiers; no core forks	Per-tenant branches in the cloud; in-process plugins	Branches would bring back 310 builds and end multi-tenancy. In-process plugins let one customer's code crash or read another's data.	Some bespoke behaviour does not fit at first; those tenants move in later waves.
Migration approach	Per-tenant waves with a one-night cutover	Big-bang; two-way sync with both systems live	Big-bang risks every customer at once. Two-way sync of financial data creates exactly the reconciliation problems we are trying to remove.	Two platforms run in parallel for about 20 months.
Rollback	Reverse ETL, rehearsed at every G1, during a hypercare window of up to 35 days	Restore the legacy snapshot only	A snapshot restore loses everything the customer entered in the cloud.	Building and testing a reverse path that is rarely used.

Decision	Chosen	Main alternatives	Why the chosen option wins	Cost we accept
AI in modernisation	AI drafts, deterministic tools verify, people approve	Rule-based VB-to-C# converters; no AI	Converters handle syntax, not the move from CSLA to a domain model. Without AI, the inventory, translations and tests are written by hand, which the 12-month window cannot absorb. The 60 percent acceptance target is calibrated on the first module.	Review capacity becomes the bottleneck and is planned as such.

Business impact

The programme meets all five goals for about 3.95M EUR over two years. From year 3 it saves 2.2M EUR a year in cash and frees about 10 engineers for product work. It pays back in month 39 on cash alone, or month 35 counting freed capacity. Five-year net cash is 3.96M EUR, and NPV at 8 percent is 2.4M EUR. All figures are indicative, built from assumptions A2, A6 and A12, and are re-baselined in week 4.

Goals against plan

Business goal	Target	What the plan delivers	Main lever
Multi-tenant cloud product	Within 12 months	First customers live in month 5; all 9 modules by month 9	Modular monolith, L0 to L3 factory

Business goal	Target	What the plan delivers	Main lever
Run cost per customer	Down 45%, to 406 EUR or less	330 EUR per migrated customer at month 12 (down 55%); 299 EUR across all customers from month 21 (down 59%)	Pooled tenancy in cells
Statutory turnaround	Under 3 weeks	20 days at most from the first cloud pack change; median 14 days	Rule packs outside the release train
Customers migrated	70% (2,940) in 12 months	2,970 (70.7%) by month 12; 100% by month 18	Waves with automated gates
Financial correctness	Zero regression	100% differential pass is a release gate; reconciliation to the cent at every cutover; signed deviation register	L3 harness, G1 and G3 gates

Costs and benefits by year

The model behind these figures works month by month from four inputs:

- **Cloud run cost:** 7 cells at full scale, a dedicated database for 3 percent of tenants, shared services and AI inference.
- **Legacy run cost:** 20 percent fixed until month 20; the variable part falls as each database is archived two months after cutover.
- **Support:** migrated tenants raise 40 percent fewer tickets at 12 EUR each.
- **Investment:** 32 people for 12 months and 10 for six more at a blended 7,300 EUR a month (3.24M), tooling and AI usage for modernisation (0.25M), and 15 percent contingency on year 1 (0.46M). The firm's own staff on the programme are existing payroll and are not counted.

The cloud inputs are planning estimates from list prices, validated against a real bill in week 4:

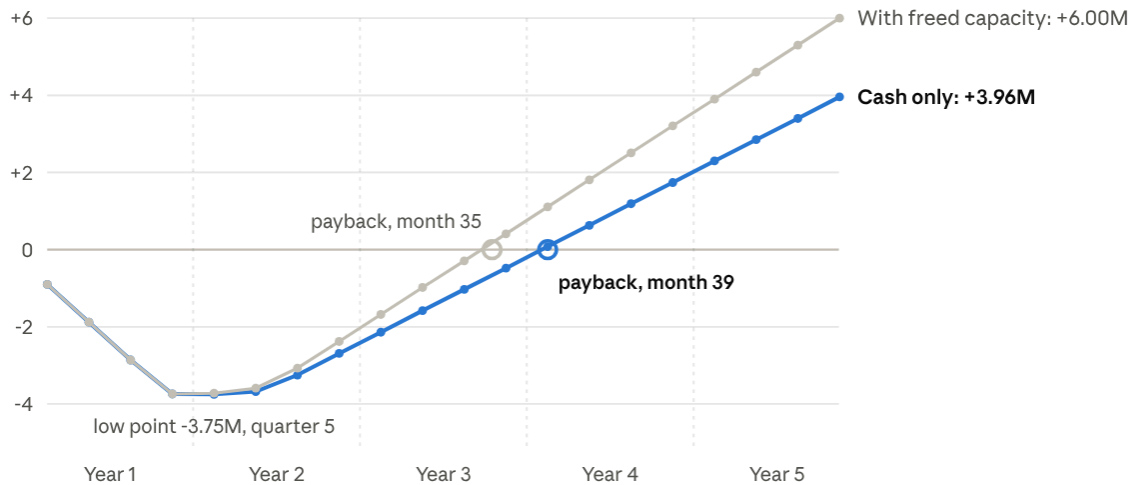
Cloud cost item	Unit cost	Quantity at full scale
Cell database: PostgreSQL Flexible Server, 16 vCores, zone-redundant, with a read replica	3,500 EUR a month	7 cells

Cloud cost item	Unit cost	Quantity at full scale
Cell compute: application containers and workers	2,000 EUR a month	7 cells
Cell storage, backup and network	800 EUR a month	7 cells
Dedicated database	150 EUR a month	about 126 tenants
Shared: edge, identity, observability, control plane, non-production environments	25,000 EUR a month	1
AI inference and document extraction	4 EUR per tenant a month	4,200 tenants
Total		1.26M EUR a year, or 299 EUR per customer

Million EUR	Year 1	Year 2	Year 3	Year 4	Year 5
Cloud run cost	0.46	1.20	1.26	1.26	1.26
Legacy run cost	2.90	0.74	0	0	0
Infrastructure saving against 3.1	-0.26	1.16	1.84	1.84	1.84
Support saving	0.03	0.33	0.37	0.37	0.37
Programme investment	3.51	0.44	0	0	0
Net cash	-3.74	1.05	2.22	2.22	2.22
Freed engineering capacity (not cash)	0	0.31	0.58	0.58	0.58

The programme pays back in month 39 on cash, month 35 counting freed capacity

Cumulative net benefit in million EUR, by quarter, base case



Business model in this document · quarterly cumulative net benefit, years 1 to 5

The low point, about 3.8M EUR, comes early in year 2, once the build is paid for. From then savings outrun the cost of running two platforms, and the curve climbs about 0.55M EUR a quarter. Figures are rounded, so table rows may not add exactly.

Sensitivity

Case	Payback (cash)	Five-year net cash
Base case	Month 39	3.96M EUR
Cloud run cost 35% higher, landing exactly at the 406 EUR target	Month 47	2.06M EUR
Waves slip by three months	Month 44	3.09M EUR
Delivery team costs 25% more	Month 44	3.04M EUR

Outcomes beyond the cash

- **Compliance.** Statutory changes stop missing the 6-week window. Today they take 10 to 14 weeks, which leaves customers filing by hand, filing late or raising tickets.
- **Engineering.** Maintenance falls from 62 percent of capacity to 30 percent or less by month 24: one code base, no fork builds, automated regression. That is about 10 engineers back on the roadmap.
- **Support.** Tickets fall from 6,500 to about 3,900 a month, driven by the grounded assistant, the end of fork-specific defects and on-time statutory changes.

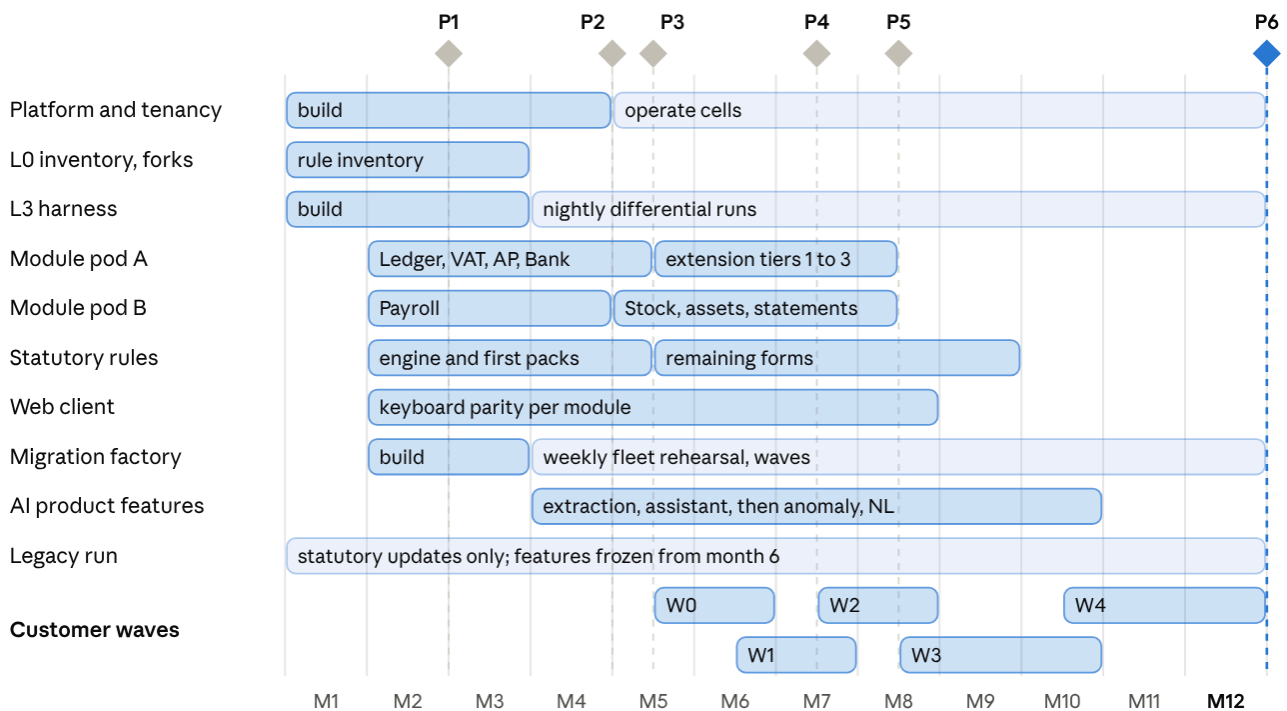
- **Scale.** A new customer adds about 230 EUR a year to cloud run cost, against 590 EUR of variable cost on legacy. Growth stops buying servers.
- **Upside not counted:** a paid AI tier (touchless invoices, NL reporting), customer growth on the cheaper platform, and switching legacy off before month 20 if the long tail moves early.

Roadmap, team and risks

The 12 months run as ten parallel workstreams through six programme gates, P1 to P6. The first customers go live in month 5 and the 70 percent mark is reached in month 12. Each gate has written criteria, and the steering committee can stop or re-sequence at any of them.

First customers in month 5, 70% by month 12, through six programme gates

Workstreams by month; darker bars build, lighter bars run; diamonds are programme gates P1 to P6



12-month roadmap · 10 workstreams, 5 waves, 6 programme gates

The L0 inventory, the L3 harness and the platform start in month 1 because everything else is checked against them. Module pod A finishes the core modules first so W0 can start in month 5, then builds the extension tiers. Pod B builds payroll first, so shadow payrolls run in months 5 and 6 before W2. The legacy lane is the cost of coexistence: statutory updates continue there until month 18.

Gates

Gate	When	Pass criteria
P1 Baseline	Week 8	Assumptions A1 to A15 confirmed or revised; L0 precision at 95% or more on Ledger and VAT; headless legacy harness running; tenancy design approved; pod velocity measured
P2 Platform ready	Month 4	Two cells live; identity, row-level security and isolation probes in place; external penetration test passed; weekly fleet rehearsal running
P3 First customers (W0)	Mid month 5	Ledger, Invoicing and VAT, Payables, Bank import and VAT e-filing pass L3; keyboard parity on their workflows; payroll engine and packs ready for shadow runs
P4 Payroll (W2)	Mid month 7	Shadow payrolls for months 5 and 6 match for every W2 tenant; Sodra and income tax packs accepted by the authorities' test services
P5 Full scope (W3)	Mid month 8	All nine modules pass L3; extension tiers 1 to 3 live
P6 70 percent	Month 12	2,940 or more tenants past G3; guardrails green; legacy end-of-life plan confirmed

Team shape

Persistent brings 32 people for 12 months, then 10 for months 13 to 18 to finish W5, hypercare and handover. Each pod owns an outcome, and pod sizes are checked against measured velocity at P1. About ten of the firm's engineers sit inside the pods from day one, so the firm runs the platform alone after month 18.

Pod	Persistent	From the firm	Owens
Programme lead (forward-deployed)	1	CTO delegate	Architecture, gates, steering
Platform and SRE	5	2	Cells, tenancy, identity, CI/CD, observability
Module pod A	5	2	Ledger, Invoicing and VAT, Payables, Bank import

Pod	Persistent	From the firm	Owns
Module pod B	5	2	Payroll, Inventory, Fixed assets, Statements
Statutory desk	1	2 analysts	Rule engine and rule packs
Web client and UX	5	1	Keyboard-first client, parity tests
Migration factory (L2)	4	1	Pipeline, reconciliation, runbook automation
Quality and equivalence (L3)	3	1	Headless harness, corpora, deviation register
AI	3	1	L0 and L1 tooling, product AI features
Domain authority	none	4 legacy engineers, 1 to 2 chartered accountants, product owner	Rule review, deviation sign-off, acceptance
Legacy run	none	2 engineers	Statutory updates on legacy until month 18
Customer migration	none	Customer success, growing to 6	Scheduling, training, hypercare

Top risks

Risk	Likelihood	Impact	Mitigation	Early warning
Knowledge sits with four engineers	High	High	L0 inventory; their review time protected; Persistent takes over their maintenance from week 2; recorded walkthroughs	Review queue older than 5 days
Hidden calculation differences (rounding, ordering)	High	High	Four L3 corpora; deviation register; history never recalculated	Diff clusters not shrinking week on week

Risk	Likelihood	Impact	Mitigation	Early warning
Accountants reject the web client	Medium	High	Parity gate; classic key map; W0 champions; training; legacy stays available until hypercare exit	Task time against legacy; W0 and W1 satisfaction
Forks do not fit the tiers (A7 wrong)	Medium	Medium	Fork analysis by week 6; misfits go to W5 and the product backlog	More than 20% of forks with unmapped changes
Statutory change mid-programme	High	Medium	Pack first in the cloud; ring-fenced legacy team; both shipped from one analysis	Change notices in the regulatory feed
Data quality across 4,200 databases	High	Medium	Weekly fleet rehearsal from month 4; pre-flight repair scripts	Share of rehearsals failing
Cross-tenant data exposure	Low	Very high	Row-level security plus application filter; hourly probes; penetration tests; per-tenant keys	Any probe failure is a Sev1
AI output below plan	Medium	Medium	Thresholds with fallback to engineer-led work; contingency budget	Acceptance under 40%
Migration throughput	Medium	High	Automated factory with 1,000 a month of capacity; self-service scheduling	Backlog of scheduled tenants growing

Appendix

Code evidence

Paths are relative to `Source/` in the [Apskaita5 repository](#), read at commit `2320f1c`.

Fact	Where to look
Schema: 67 tables, 829 columns, DOUBLE money, no foreign keys	<code>Apskaita5/Apskaita5/DbStructure/DatabaseStructureGauge.xml</code>
VB rounding helper (compares through Decimal)	<code>Apskaita5/AccCommon/CommonMethods.vb</code> , lines 14 to 24
MySQL <code>CROUND</code> stored function (compares in DOUBLE)	<code>DatabaseStructureGauge.xml</code> , stored procedure list
Named SQL dictionary, 770 MySQL and 761 SQLite statements	<code>AccDataAccessLayer/AccDataAccessLayer/SqlDepositories/*.sqld</code>
Schema compare and fix; old-version sniffing	<code>AccDataAccessLayer/AccDataAccessLayer/DatabaseAccess/DatabaseStructure/DatabaseStructureErrorList.vb</code> ; <code>Apskaita5/Apskaita5/CustomDatabaseStructureErrorManager.vb</code>
One class per declaration version, hardcoded validity dates	<code>Apskaita5/Apskaita5/ActiveReports/Declarations/</code> (e.g. <code>DeclarationSAM_7.vb</code>)
Declaration list hardcoded in the forms layer	<code>Apskaita5/AccDataBindingsWinForms/CommonMethods.vb</code> , lines 577 to 621
Hardcoded 2019 social-insurance gross-up of 1.289	<code>Apskaita5/Apskaita5/ActiveReports/WageVDUInfo.vb</code> , line 151
Per-company editable non-taxable income formula	<code>Apskaita5/Apskaita5/General/CompanyObjects/Company.vb</code> , line 49
Global company context, 321 uses	<code>GetCurrentCompany()</code> in <code>Apskaita5/Apskaita5/Utilities.vb</code>
Grid keyboard behaviour (Insert, Delete, Enter, Tab)	<code>Apskaita5/AccDataBindingsWinForms/DataListViewEditControlManager.vb</code> , lines 1260 to 1300
Date shorthand entry	<code>Apskaita5/AccControlsWinForms/Controls/AccDatePicker.vb</code> , lines 310 to 340
Positional report data set, 345 columns	<code>Apskaita5/AccControlsWinForms/ReportData.xsd</code>
Plaintext database credential	<code>Apskaita5/AccWebService/Web.config</code> , line 58
Frameworks: .NET 2.0 and 3.5, CSLA 2.1.4	each <code>*.vbproj</code> ; <code>ExternalReferencedLibraries/</code>

The rounding disagreement was found by reading both implementations and emulating them over all one million half-cent values from 0.005 to 9,999.995. 6.6 percent of them round differently, and so do 354 of 200,000 random 21 percent VAT calculations. The L3 harness confirms this on the real runtimes in weeks 2 to 4.

Glossary

Term	Meaning
VMI	Lithuanian State Tax Inspectorate
Sodra	Lithuanian State Social Insurance Fund
GPM, PSD, NPD	Personal income tax; compulsory health insurance; non-taxable income amount
FR0600, i.SAF, SAF-T	VAT return; monthly VAT invoice register file; standard audit file for tax
SAM, 13-SD, GPM313	Sodra monthly and event declarations; monthly income tax declaration
FFData, MXFD	Data file and form-definition file of the e-form software used for Lithuanian declarations
camt.052, camt.053, pain.001	ISO 20022 bank statement and payment initiation messages
LITAS-ESIS	Legacy Lithuanian bank statement format
CSLA	.NET business-object framework used by the legacy code
Data-portal method	CSLA method that loads or saves a business object (fetch, insert, update, delete)
Ambient transaction	One database transaction shared implicitly by every save inside it
Row-level security (RLS)	Database rule that filters every row by tenant for every query
Cell	Self-contained slice of the platform serving about 600 tenants
Transactional outbox	Events written in the same database transaction as the data, then published, so none are lost

Term	Meaning
Aggregate	A group of objects saved together as one unit, such as an invoice with its lines
UUIDv7	A globally unique key that sorts by creation time
Crypto-shredding	Making data unreadable by deleting its encryption key
WAF	Web application firewall at the edge
Semantic layer	Catalogue of business metrics with fixed definitions that queries must use
Calibrated confidence	A model score tuned so that 0.98 really means right 98 times in 100
Canonicalisation	Normalising output (ordering, whitespace, formats) before comparing it
Sev1, Sev2	Highest severities: a correctness or outage defect, and a major defect with a workaround
G0 to G4	Per-tenant migration gates, from eligibility to hypercare exit
P1 to P6	Programme gates on the 12-month roadmap
Rule pack	Signed, effective-dated bundle of statutory parameters, calculations, filing definitions and tests
Deviation register	Signed list of intended differences from legacy behaviour, each with its reason